AIML B25 – PGCP

Project 6 : CodeGen

Group -12(CodeGen3)

Anant Joshi
Srivalli Janapati
Sai Madishetty
Saurabh Sinha

Project Mentor: Nayan Jha
Project Supervisor: Anil Nelakanti
January 24-25, 2026

Saturday, 24 January 2026

Table of Contents

1. Abstract

This project focuses on the problem of translating natural language and structured queries into executable database queries across different database paradigms. Two related tasks are addressed: converting natural language questions into SQL queries (Text-to-SQL) and translating SQL queries into equivalent MongoDB aggregation pipelines (SQL-to-NoSQL). Traditional neural approaches such as sequence-based and transformer-based models are evaluated alongside large language model (LLM)-based methods. To improve semantic correctness, the project emphasizes execution-based validation rather than syntactic matching. Schema-aware retrieval-augmented generation (RAG) and execution-guided feedback are explored to reduce hallucinations and improve correctness in LLM-generated queries. Experimental results demonstrate that relation-aware models outperform sequential baselines for Text-to-SQL, while schema-grounded LLMs significantly improve SQL-to-NoSQL translation accuracy.

2. Introduction

Databases remain the backbone of modern data-driven systems, and structured query languages such as SQL and MongoDB's aggregation framework are essential for retrieving and analyzing data. However, writing correct queries requires technical expertise, making database interaction challenging for non-technical users. Natural language interfaces aim to bridge this gap by automatically generating structured queries from user questions.

Despite recent advances, generating correct queries remains difficult due to the need for schema understanding, relational reasoning, and strict execution semantics. A syntactically valid query may still produce incorrect results if joins, filters, or aggregations are misinterpreted. With the rise of large language models, there is renewed interest in using generative models for structured query generation. However, LLMs often hallucinate schema elements or generate queries that fail during execution. This project investigates how traditional neural models and LLM-based approaches can be combined with schema awareness and execution-guided validation to improve reliability

3. Problem Statement

3.1 Natural Language to SQL Translation

The first problem addressed in this project is the automatic generation of SQL queries from natural language questions. Given a user query and a database schema, the

system must produce an executable SQL statement that accurately reflects the user's intent. The task becomes challenging in real-world settings due to the presence of multiple tables, complex joins, nested subqueries, grouping, and aggregation operations. Additionally, models must generalize across unseen schemas rather than memorizing database-specific patterns.

3.2 SQL to NoSQL Translation

The second problem involves translating SQL queries into equivalent MongoDB aggregation pipelines. Relational databases and document-oriented databases follow fundamentally different data models. While SQL relies on joins and normalized tables, MongoDB requires explicit aggregation stages such as lookups and unwinding of documents. The objective is to preserve the semantics of the original SQL query so that the MongoDB pipeline produces the same result set when executed on equivalent data

4. Project Objectives

The primary objectives of this project are:

- To study and benchmark traditional Text-to-SQL models using sequence-based, transformer-based, and relation-aware architectures.
- To analyze the limitations of schema-agnostic approaches for structured query generation.
- To design and evaluate SQL-to-NoSQL translation pipelines using both rule-based and learning-based methods.
- To explore schema-aware retrieval-augmented generation for grounding LLM outputs.
- To evaluate models using execution-based metrics that reflect real-world correctness.

5. Background

5.1 Traditional Text-to-SQL Approaches

Early Text-to-SQL systems relied on sequence-to-sequence models, typically using LSTM encoders and decoders. In these approaches, both the natural language question and database schema are represented as flattened text sequences. While such models can generate simple SQL queries, they struggle with complex multi-table joins and nested queries due to their limited ability to model relational structure.

5.2 Transformer-Based Models

Transformer architectures introduced attention mechanisms that improved alignment between user questions and schema elements. By jointly encoding the question and schema, transformer-based models achieve better performance than sequence-only approaches. However, standard transformers still face challenges in explicitly modeling relationships such as primary-foreign key dependencies.

5.3 Relation-Aware Models

Relation-aware models address this limitation by representing the schema as a graph. Tables and columns are modeled as nodes, and relationships such as foreign keys are encoded as edges. This representation enables the model to reason explicitly about joins and table relationships, leading to improved performance on complex queries.

5.4 LLM-Based Structured Query Generation

Large language models have recently been applied to structured query generation using prompt-based techniques. While LLMs demonstrate strong language understanding,

they often generate invalid or non-executable queries when schema context is missing. This has motivated the use of schema grounding, retrieval-augmented generation, and execution-guided feedback to improve reliability.

6. Methodology

6.1 Overall Approach

The overall architecture of the project is designed to support two structured translation tasks: Natural Language to SQL and SQL to NoSQL. The system follows a modular pipeline where each model operates as an independent processing unit, allowing comparison across different architectural paradigms. The architecture can be broadly divided into three layers: input processing, model inference, and execution-based validation.

At the input layer, user queries and database schemas are preprocessed and converted into model-specific representations. The inference layer contains multiple model pipelines, including traditional neural models and large language model–based systems. Finally, the output layer validates generated queries using database execution to ensure semantic correctness.

6.2 Text-to-SQL Methodology

The Text-to-SQL pipeline begins with a natural language question and the corresponding database schema. Depending on the model, the schema is represented either as flattened text or as a structured relational graph.

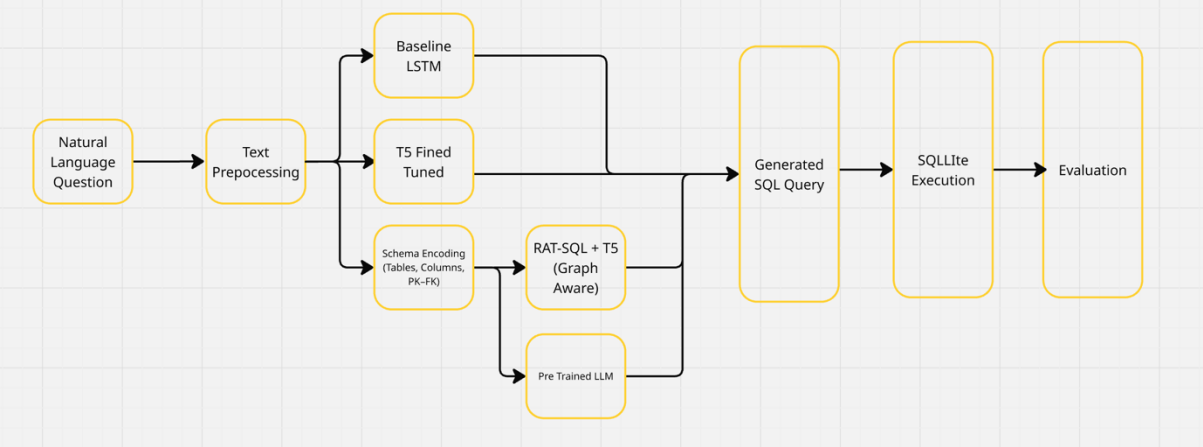
In sequence-based models, the question and schema are tokenized and encoded using recurrent neural networks. The decoder generates SQL queries token by token, relying solely on learned sequence patterns. Transformer-based models improve upon this by applying self-attention mechanisms that allow the model to align relevant schema elements with parts of the user query.

Relation-aware models further extend this architecture by constructing a graph representation of the schema. Tables and columns are treated as nodes, while primary key and foreign key relationships are encoded as edges. A relation-aware transformer encoder processes this graph along with the question tokens, enabling explicit modeling of joins and table relationships. The decoder then generates SQL queries that better reflect the relational structure of the database.

Process Flow Summary:

1. User question and schema ingestion
2. Schema representation (textual or graph-based)
3. Encoder processing (LSTM / Transformer / Relation-aware Transformer)

4. Autoregressive SQL generation
5. SQL execution for validation



6.3 Traditional SQL-to-NoSQL Methodology

Traditional SQL-to-NoSQL translation approaches rely on deterministic or learning-based mappings to convert relational queries into MongoDB aggregation pipelines. In rule-based systems, SQL queries are first parsed into an intermediate representation, such as an abstract syntax tree (AST). Each SQL construct, including SELECT, WHERE, GROUP BY, and JOIN clauses, is then mapped to corresponding MongoDB aggregation stages such as $project, $match, $group, and $lookup.

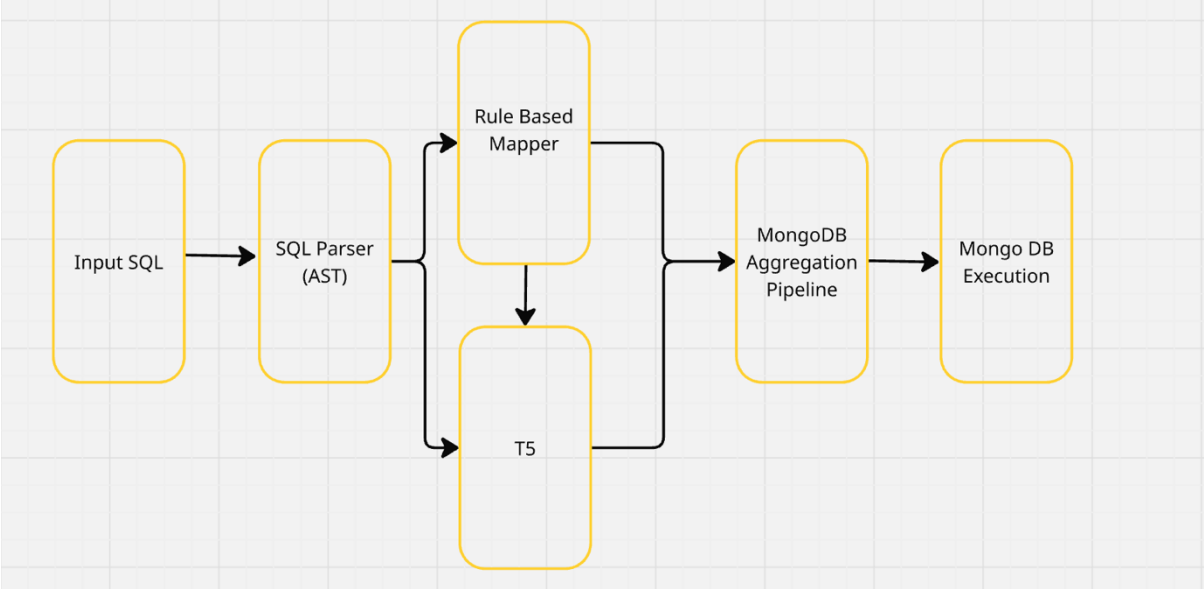
While this approach ensures syntactic correctness for simple queries, it lacks flexibility and scalability. Complex queries involving nested subqueries, multiple joins, or conditional aggregations require handcrafted rules, which are difficult to maintain and extend. Furthermore, rule-based methods assume fixed schema knowledge and fail to generalize across different database structures.

Learning-based traditional methods treat SQL-to-NoSQL translation as a sequence-to-sequence problem. In this architecture, SQL queries are tokenized and encoded using recurrent or transformer-based encoders, and MongoDB pipelines are generated autoregressively. Although these models reduce manual rule engineering, they still struggle without explicit schema awareness. As a result, generated pipelines often omit necessary join operations or reference non-existent collections and fields.

Process Flow (Traditional Methods):

1. SQL query input
2. SQL parsing into AST or token sequence
3. Rule-based mapping or sequence-based generation
4. MongoDB aggregation pipeline generation

5. Optional execution for validation



6.4 LLM-Based SQL-to-NoSQL with RAG

To overcome the limitations of traditional approaches, this project introduces a large language model–based SQL-to-NoSQL translation pipeline augmented with retrieval-augmented generation (RAG). Unlike traditional models, this approach explicitly grounds generation in database-specific schema information.

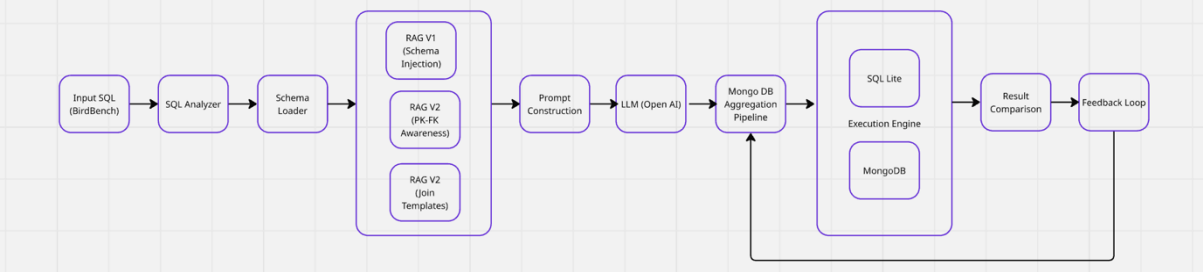
The pipeline begins with the extraction of relational schema metadata, including table names, column names, data types, and primary–foreign key relationships. This schema information is stored in a retrievable format and dynamically injected into the LLM prompt at inference time. By providing schema context, the model is guided toward generating MongoDB pipelines that reflect the actual database structure.

For queries involving joins, predefined aggregation templates are automatically constructed. These templates include $lookup and $unwind stages corresponding to relational join paths derived from foreign key relationships. The templates are included in the prompt to operationalize join semantics, reducing the likelihood of missing or incorrect join stages.

After pipeline generation, the output is executed against a MongoDB instance. The execution result is compared with the output of the original SQL query. If discrepancies or execution failures occur, corrective feedback is fed back to the LLM, enabling iterative refinement. This execution-guided feedback loop significantly improves semantic correctness and reduces hallucinations.

Process Flow (LLM + RAG Method):

1. SQL query input
2. Schema and relationship extraction
3. Retrieval of schema context and join templates
4. Prompt augmentation with retrieved context
5. LLM-based MongoDB pipeline generation
6. Execution on MongoDB
7. Result comparison and feedback-driven correction



6.5 Model Architectures

Model	Architecture Type	Schema Representation	Approx. Parameter Scale	Strengths	Limitations
SQLNet	LSTM Encoder–Decoder	Flattened text	Low (Millions)	Simple, efficient	Poor relational reasoning
T5 Base	Transformer Encoder–Decoder	Text-based schema	~220M	Strong NL understanding	No explicit relations
RAT-SQL-T5-Lite	Relation-aware Transformer	Graph-based schema	Moderate	Explicit join reasoning	Higher preprocessing complexity
GaussAlgo (T5-Large)	Deep Transformer	Text-based schema	~770M	Strong generation ability	Relational errors
LLM + RAG	Large Language Model	Retrieved schema context	Billions	Flexible, extensible	High cost, execution dependency

7. Datasets Used

The evaluation of structured query generation models requires datasets that capture varying levels of query complexity, schema diversity, and execution semantics. To ensure comprehensive analysis, this project uses multiple datasets that together represent both simplified academic benchmarks and realistic production-oriented workloads.

7.1 Spider Dataset

The Spider dataset is a large-scale, cross-domain Text-to-SQL benchmark designed to evaluate a model's ability to generalize across unseen database schemas. It contains hundreds of databases spanning diverse domains, each with distinct schemas and table relationships. Queries in Spider frequently involve complex SQL constructs such as multi-table joins, nested subqueries, grouping, aggregation, and ordering.

Unlike earlier benchmarks, Spider explicitly separates training and test databases, preventing models from memorizing schema-specific patterns. This characteristic makes Spider particularly suitable for evaluating schema understanding and relational reasoning capabilities. Due to its complexity and realism, Spider has become the standard benchmark for modern Text-to-SQL research and is widely used to evaluate transformer-based and relation-aware models.

In this project, Spider is used to evaluate the effectiveness of sequence-based, transformer-based, and graph-based Text-to-SQL models, with an emphasis on execution accuracy rather than exact string matching.

7.2 WikiSQL Dataset

The WikiSQL dataset represents a simpler Text-to-SQL benchmark consisting primarily of single-table queries. Queries typically involve basic SELECT, WHERE, and aggregation operations without joins, nested queries, or grouping across multiple tables.

Although WikiSQL does not reflect the complexity of real-world databases, it serves an important role as a baseline dataset. It allows evaluation of a model's ability to handle straightforward query translation and helps isolate improvements attributable to model architecture rather than schema complexity.

In this project, WikiSQL is used to validate baseline performance and to contrast model behavior between simple and complex query settings. The dataset highlights the limitations of models that perform well on simple queries but fail to generalize to multi-table scenarios.

7.3 BirdBench Dataset

BirdBench is an execution-centric benchmark designed to evaluate semantic correctness rather than syntactic similarity. Unlike traditional Text-to-SQL datasets, BirdBench focuses on validating query outputs by executing them on real database engines and comparing results. It supports multiple database systems, including SQLite, PostgreSQL, and MySQL, and includes queries that reflect real-world analytical workloads.

BirdBench introduces realistic challenges such as type mismatches, engine-specific behavior, date handling, and complex join conditions. By evaluating the full pipeline from natural language to database execution, BirdBench exposes failure modes that are often hidden by string-based evaluation metrics.

In this project, BirdBench plays a critical role in evaluating SQL-to-NoSQL translation. Ground truth SQL queries from BirdBench are used as input for SQL-to-NoSQL translation, and the generated MongoDB pipelines are validated by comparing execution results across relational and document-based databases. This makes BirdBench particularly suitable for assessing semantic preservation and execution equivalence.

7.4 Dataset Selection Rationale

The combination of Spider, WikiSQL, and BirdBench enables a balanced evaluation across different dimensions of structured query generation. WikiSQL provides a baseline for simple query translation, Spider evaluates complex cross-schema generalization, and BirdBench ensures execution-level correctness in realistic environments. Together, these datasets allow the project to assess both model capability and practical applicability, aligning experimental evaluation with real-world database usage scenarios.

8. Evaluation Metrics

8.1 Evaluation Metrics for Text-to-SQL

- **Exact Match Accuracy**
  Exact match accuracy measures whether the generated SQL query exactly matches the reference SQL query at the token level. While this metric is simple to compute, it is overly strict and fails to account for semantically equivalent queries that differ syntactically, such as reordered joins or alternative filtering conditions.
- **Execution Accuracy**
  Execution accuracy evaluates whether the generated SQL query produces the same result as the reference query when executed on the database. This metric

directly captures semantic correctness and is more reliable than exact match accuracy, especially for complex queries.

- **Test Suite Accuracy**
  Test suite accuracy extends execution accuracy by evaluating generated queries across multiple database variations. A query is considered correct only if it produces the same output as the reference query on all test databases. This metric helps identify queries that overfit to specific database instances.
- **Valid Efficiency Score**
  The valid efficiency score combines correctness and execution performance. Only valid queries are scored, and among them, faster queries receive higher scores. This metric encourages models to generate not only correct but also efficient SQL queries.

8.2 Evaluation Metrics for SQL-to-NoSQL

- Pipeline Structure Match
  This metric compares the generated MongoDB aggregation pipeline with a reference pipeline to assess structural similarity. It evaluates whether the correct sequence of aggregation stages is used, although it does not guarantee semantic equivalence.
- Execution Accuracy
  Execution accuracy measures whether the MongoDB pipeline returns the same result set as the original SQL query. This is the primary correctness metric for SQL-to-NoSQL translation, as it directly validates semantic preservation across database paradigms.
- Efficiency Ratio
  The efficiency ratio compares the execution time of the generated pipeline against a reference pipeline. A higher ratio indicates a more efficient translation. This metric helps identify pipelines that are functionally correct but suboptimal in performance.

8.2 Evaluation Results

Text 2 SQL

Model	Exact Match Accuracy	Execution Accuracy	Test Suite Accuracy	Valid Efficiency Score
SQLNet (LSTM Baseline)	0.01 (1%)	0.01 (1%)	0.02 (2%)	0.001 (0.1%)
T5 Fine-Tuned (Spider)	0.24 (24%)	0.38 (38%)	0.38 (38%)	0.031 (3.1%)
GaussAlgo (T5-Large)	0.15 (15%)	0.41 (41%)	0.41 (41%)	0.020 (2.0%)
RAT-SQL-T5-Lite	0.29 (29%)	0.42 (42%)	0.42 (42%)	0.035 (3.5%)

The LSTM-based SQLNet model performs poorly across all evaluation metrics. Its low execution accuracy highlights the inability of sequence-only models to capture complex schema relationships, particularly in multi-table queries. This confirms the limitations of early Text-to-SQL approaches when applied to realistic datasets.

Transformer-based models demonstrate substantial improvements. The fine-tuned T5 model shows a significant increase in execution accuracy, indicating that attention-based mechanisms enable better alignment between user questions and schema elements. The GaussAlgo T5-Large model further improves execution accuracy, benefiting from a deeper transformer architecture and stronger pretraining.

The best overall performance is achieved by the relation-aware RAT-SQL-T5-Lite model. By explicitly modeling schema relationships using graph-based representations, this model generates more accurate joins and aggregations. The higher test suite accuracy indicates stronger generalization across database variations, while the valid efficiency score suggests that the generated queries are not only correct but also relatively efficient.

SQL-to-NoSQL (Traditional Methods)

Approach	Observations
Rule-Based Translation	Performs well for simple queries; fails on joins, nested queries, and complex aggregations
T5 Learning-Based (No Schema)	Generates syntactically plausible pipelines but frequently misses $lookup stages or references invalid fields

Rule-based approaches demonstrate predictable behavior for simple queries but fail to scale as query complexity increases. Maintaining handcrafted rules for all SQL constructs is impractical, and these methods lack robustness across schemas.

Learning-based models without schema grounding perform marginally better but still suffer from hallucinated collections and incomplete pipelines. These results indicate that SQL-to-NoSQL translation cannot be reliably solved without explicit schema awareness

SQL-to-NoSQL Evaluation Results (LLM-Based Models)

Model	Total Queries	Successful Executions	Execution Accuracy	Average Retries
LLM Baseline (No RAG)	20	1	0.05 (5%)	4
LLM + RAG (Schema Awareness)	20	5	0.25 (25%)	0.4
LLM + RAG (Schema + Join Templates)	20	6	0.30 (30%)	0.3

The baseline LLM model performs poorly, achieving very low execution accuracy. Without schema context, the model frequently hallucinates joins, omits required aggregation stages, or generates invalid MongoDB syntax. The high number of retries further indicates instability in generation.

Introducing schema-aware RAG leads to a substantial improvement in execution accuracy. By grounding the model in database structure, the number of invalid pipelines decreases significantly, and the model converges faster with fewer retries.

The best performance is achieved when schema-aware RAG is combined with join template injection. By explicitly providing $lookup and $unwind templates derived from foreign key relationships, the model generates more complete and semantically correct pipelines. This approach yields the highest execution accuracy and the lowest retry rate, demonstrating the effectiveness of combining structural grounding with execution-guided feedback

9. Limitations

Despite demonstrating promising results, the proposed approaches in this project are subject to several limitations related to data, modeling, evaluation, and system design. These limitations highlight areas where further improvements and extensions are required.

- **Dataset and Evaluation Scale** : The experimental evaluation is conducted on a limited subset of available benchmark datasets due to computational and time constraints. Although datasets such as Spider, WikiSQL, and BirdBench provide representative query complexity, the number of evaluated samples does not fully capture the diversity of real-world database workloads. As a result, the reported results should be interpreted as indicative trends rather than absolute performance guarantees.

- **Limited NoSQL Scope** : The SQL-to-NoSQL translation component focuses exclusively on MongoDB as the target NoSQL system. While MongoDB is widely used, other NoSQL databases such as Cassandra, DynamoDB, and Neo4j follow fundamentally different data models and query paradigms. The current

approach does not generalize directly to these systems without additional schema representation and translation logic.

- **Schema Dependence in LLM-Based Method :** Although schema-aware retrieval-augmented generation significantly improves performance, the LLM-based methods remain heavily dependent on the quality and completeness of retrieved schema information. Incomplete or incorrect schema metadata can still lead to invalid query generation. Furthermore, schema extraction and prompt construction introduce additional complexity into the system pipeline.

- **Limited Query Complexity Coverage** : Although the project evaluates complex queries involving joins and aggregations, certain advanced SQL constructs such as window functions, recursive queries, and database-specific extensions are not explicitly supported. As a result, the system may not handle all possible SQL formulations encountered in production environments.

- **Generalization Beyond Benchmarks** : The models are evaluated primarily on academic benchmarks and controlled datasets. Real-world databases often include noisy schemas, incomplete documentation, and evolving data distributions. The robustness of the proposed methods under such conditions has not been fully validated.

- **System Integration Limitations:** The proposed architecture is evaluated in an experimental setting and is not integrated into a production-grade database system. Factors such as concurrency control, access permissions, query optimization strategies, and security considerations are beyond the scope of this project.

10. Future Work

The outcomes of this project indicate that the proposed framework extends beyond SQL and MongoDB translation and can be generalized to a broader class of structured language transformation problems. The core idea is to decouple reasoning, structure grounding, and validation, allowing the same system design to be reused across domains

The proposed system follows a general execution-guided flow:

Structured Input → Target Representation Generation → Execution → Validation → Feedback

In this framework, the large language model is responsible for reasoning and generation, while task-specific constraints are injected through retrieval-augmented context. The execution or validation module acts as an external verifier, ensuring semantic correctness. This separation enables the framework to remain task-agnostic while adapting to different structured outputs.

Claim:
By modifying only the retrieved context and validation mechanism, the same architecture can be applied to multiple structured translation tasks without retraining the core language model.

Future work can extend the framework to additional structured domains such as:

- Natural language to API calls
- Code generation with compiler-based validation
- Configuration file generation (YAML, JSON, Terraform)
- Natural language to graph queries (e.g., Cypher)

In each case, domain-specific rules and schemas can be retrieved and injected via RAG, while execution or compilation results serve as the validation signal.

The future roadmap can be summarized as:

LLM Reasoning Core (Unchanged)
↓
Task-Specific Context via RAG (Schema, APIs, Rules)
↓
Target Language Generation
↓
Execution / Validation Engine
↓
Feedback-Driven Refinement

This flow emphasizes that the intelligence remains centralized in the LLM, while correctness and domain specificity are enforced externally.

11. Conclusion

This project explored structured query generation through two complementary tasks: Natural Language to SQL and SQL to NoSQL translation. By evaluating traditional neural models alongside large language model–based approaches, the project highlights the strengths and limitations of different paradigms in handling structured data.

The experimental results demonstrate that sequence-based models are insufficient for complex relational reasoning, while transformer-based and relation-aware architectures significantly improve Text-to-SQL performance. For SQL-to-NoSQL translation, schema-agnostic approaches fail to preserve semantic correctness, whereas schema-aware retrieval-augmented generation combined with execution-guided feedback leads to more reliable results.A key contribution of this project is the emphasis on execution-based validation as the primary correctness signal. By validating generated queries through actual database execution, the system moves beyond syntactic similarity and aligns more closely with real-world database usage. Furthermore, the proposed architecture illustrates how LLMs can be effectively grounded using external structure and feedback without requiring expensive fine-tuning.

Overall, this project demonstrates that reliable structured query generation requires a combination of language understanding, schema awareness, and execution-level verification. The proposed execution-guided, retrieval-augmented framework provides a scalable and extensible foundation for future research and practical applications in structured language translation.

12. Reference

Category	Name	Description	Reference Link
Model	CodeGen-350M-Multi	A pretrained code generation model supporting multiple programming languages, used as a reference for structured code and query generation approaches.	https://huggingface.co/Salesforce/codegen-350M-multi
Dataset	Spider	A cross-domain, cross-database Text-to-SQL benchmark containing complex SQL queries with joins, nested queries, and aggregations.	https://spider2-sql.github.io/
Dataset	BirdBench	An execution-based benchmark focusing on semantic correctness of queries across real database engines.	https://bird-bench.github.io/
Survey Paper	Review of Text-to-SQL	A comprehensive survey covering traditional, neural, and LLM-based approaches for Text-to-SQL generation.	https://arxiv.org/pdf/2410.01066
Research Paper	SQL to NoSQL Translation	A research work proposing schema-aware, execution-guided LLM-based approaches for SQL-to-NoSQL translation.	https://arxiv.org/pdf/2502.11201